```
Scenario: Purchasing items in my cart
    Given I am ready to order
    When I pay for the items in my cart
    Then I should be sent a copy of 'BDD in Action'
```

❷ **Checking out**

This is wrong on several levels. The second scenario ❷ will not work unless the first one ❶ was run directly before it, and the system remained in the same state. But the testing framework will create a fresh environment for each scenario to run in, so the cart will be empty at the start of the second scenario.

In addition, you can't guarantee the order in which the scenarios will be run, or even which scenarios will be run (you may only run a subset of the scenarios for a particular test run).

On another level, these tests are a little too granular. They focus on small chunks of the business flow, at the risk of losing view of the overall objectives. In fact, a good way to make your tests more robust is to make them very high-level. A good acceptance criterion captures the business intent of a requirement, not the mechanics of how it gets done. For example, suppose your website prides itself on being environmentally friendly and is famed for its innovative delivery methods. Your real requirement might be more accurately captured in a high-level scenario like this one:

```
Scenario: Buying an item
    Given I am looking for a good book to read
    When I purchase a copy of 'BDD in Action'
    Then I should be sent my copy by specially trained environmentally
    friendly courier pigeon
```

High-level scenarios like this give a much better picture of why you need to implement a feature, and what sort of things you'll value in it. Scenarios like this are highly maintainable, because the business scenario you're describing is much less likely to change than, for example, the user interface you'd build to deliver this feature.

Another way to avoid dependencies between scenarios is to focus on specific business rules. Not all scenarios will be high-level, end-to-end scenarios like the preceding "Buying an Item" example. Others will capture more specific business requirements such as the Frequent Flyer point calculations that you saw earlier in this chapter. Once again, the scenarios should focus on the business requirements and expected outcomes, rather than on the implementation details of how this happens.

Of course, sometimes you'll want to see the details. Don't worry: in the following chapters you'll see how to write high-level tests at this level that are both very maintainable and that still provide all the nitty-gritty low-level details for anyone who needs them.

5.5 Organizing your scenarios using feature files and tags

In real-world projects, scenarios can become quite numerous, and it's important to keep them well organized. You may also need to be able to identify and group scenarios in different ways; for example, you might want to distinguish UI-related scenarios from batch-processing scenarios, or identify the scenarios related to cross-functional